Algorithms

Lecture # 30

Syed Hamed Raza

Depth-first Traversal

Depth-first Traversal

- Assume that the graph is represented by an adjacency list so the overhead of the for loop in line 7 is constant per edge.

```
TRAVERSE(s)
 1 push(∅, s)
 2 while stack not
   empty
 3 do pop(p, v)
 4 if (v is unmarked )
 5 then mark v
 6 parent (v) ← p
 7 for each edge (v,w)
 8 do push(v,w)
```

Depth-first Traversal

Depth-first Traversal

- Assume that the graph is represented by an adjacency list so the overhead of the for loop in line 7 is constant per edge.

```
TRAVERSE(s)
 1 push(∅, s)
 2 while stack not
   empty
 3 do pop(p, v)
 4 if (v is unmarked )
 5 then mark v
 6 parent (v) ← p
 7 for each edge (v,w)
 8 do push(v,w)
```

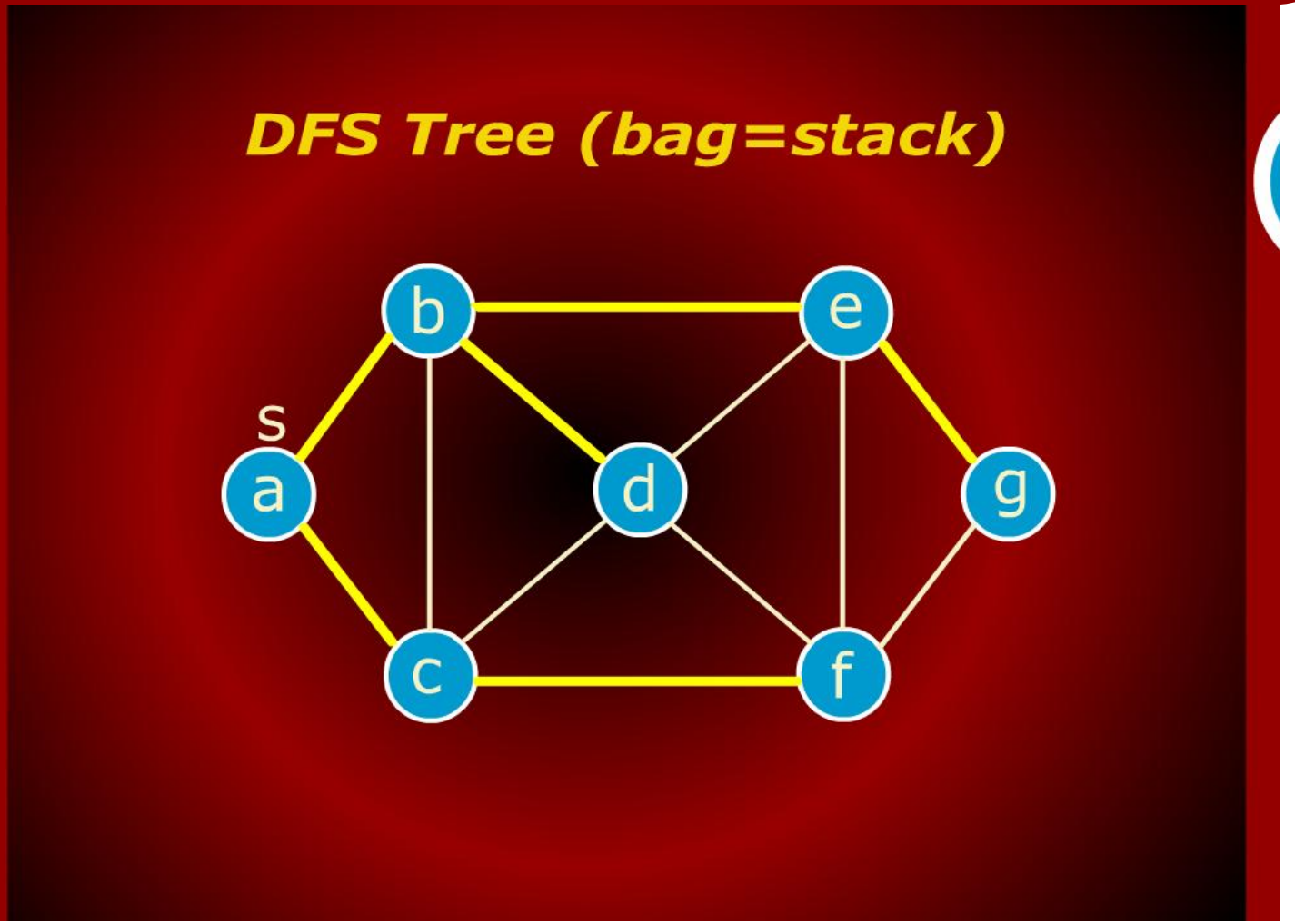
Model of Computation

Depth-first Traversal

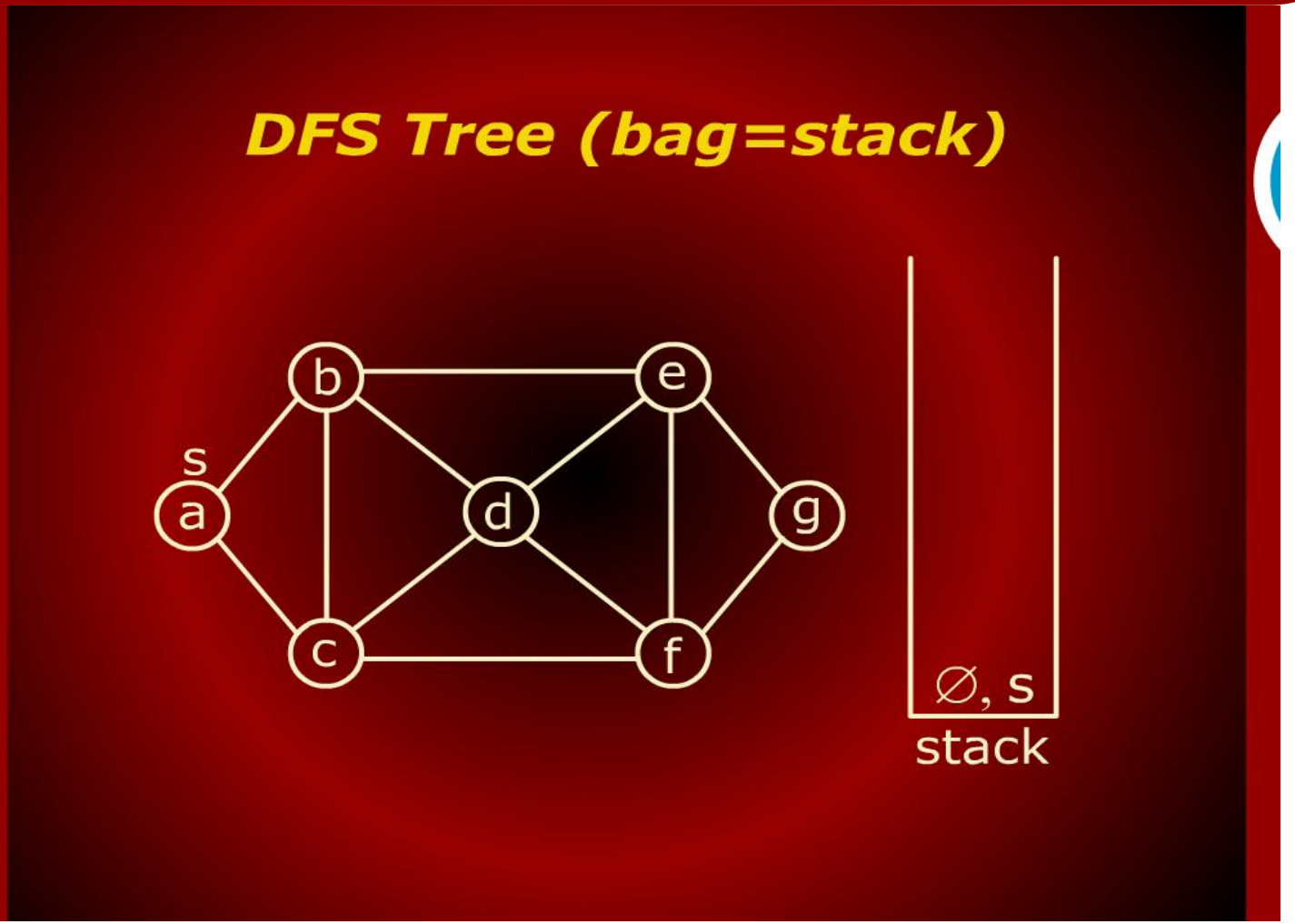
- If we implement the bag by using a stack,we have depth-first search (DFS) or traversal.

```
TRAVERSE(s)
 1 push(∅, s)
 2 while stack not
   empty
 3 do pop(p, v)
 4 if (v is unmarked )
 5 then mark v
 6 parent (v) ← p
 7 for each edge (v,w)
 8 do push(v,w)
```

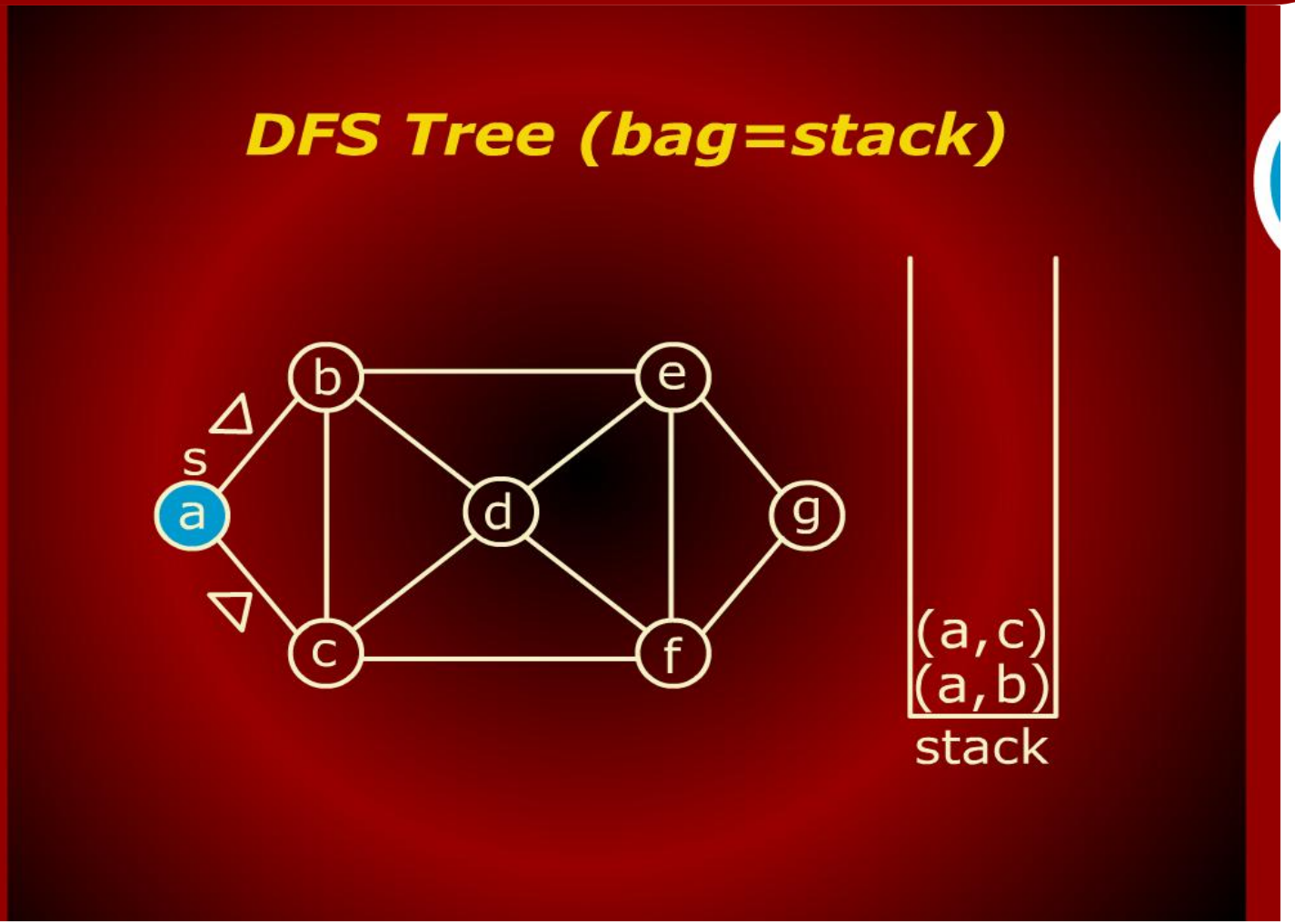
DFS Tree (bag=stack)



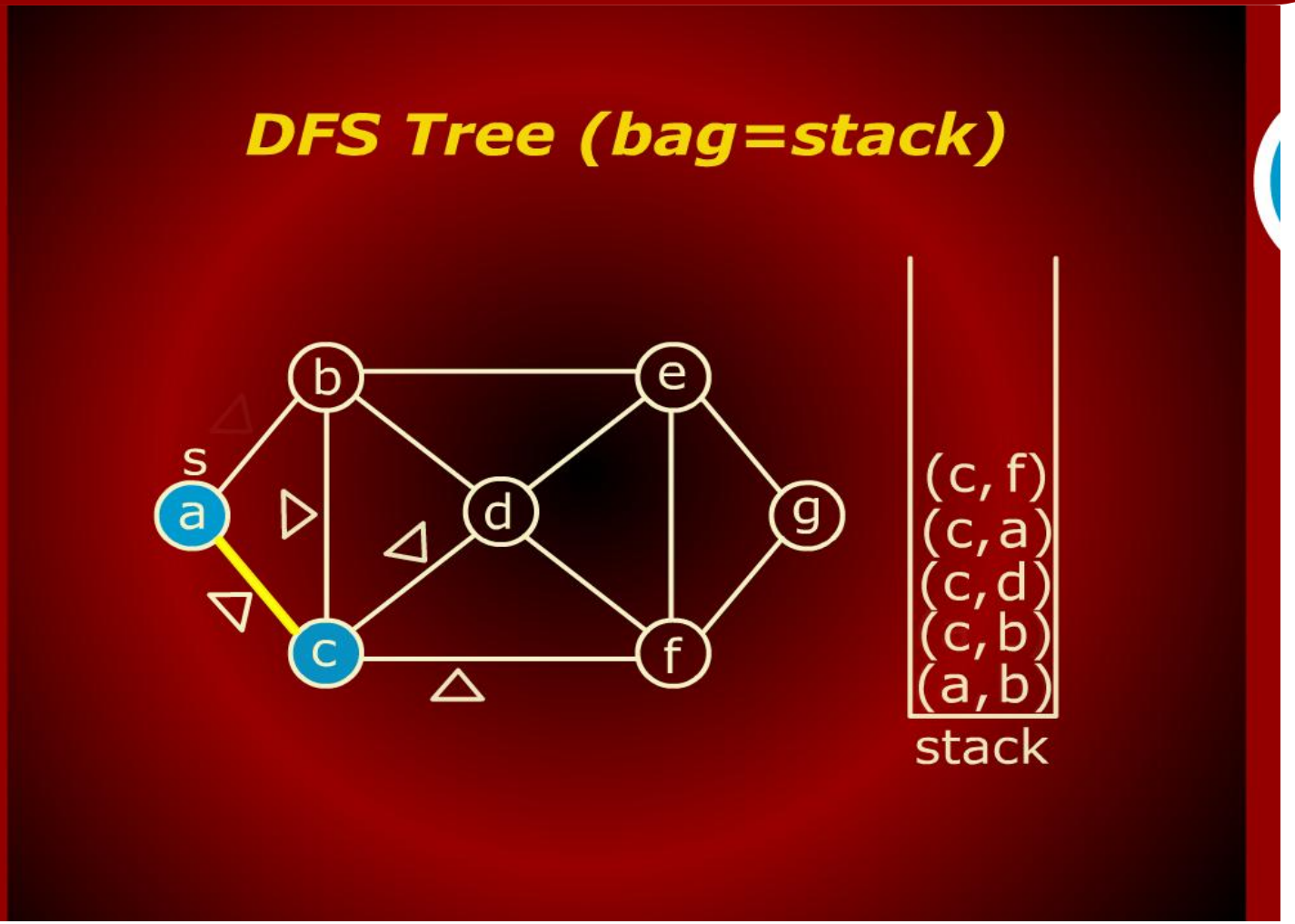
DFS Tree (bag=stack)



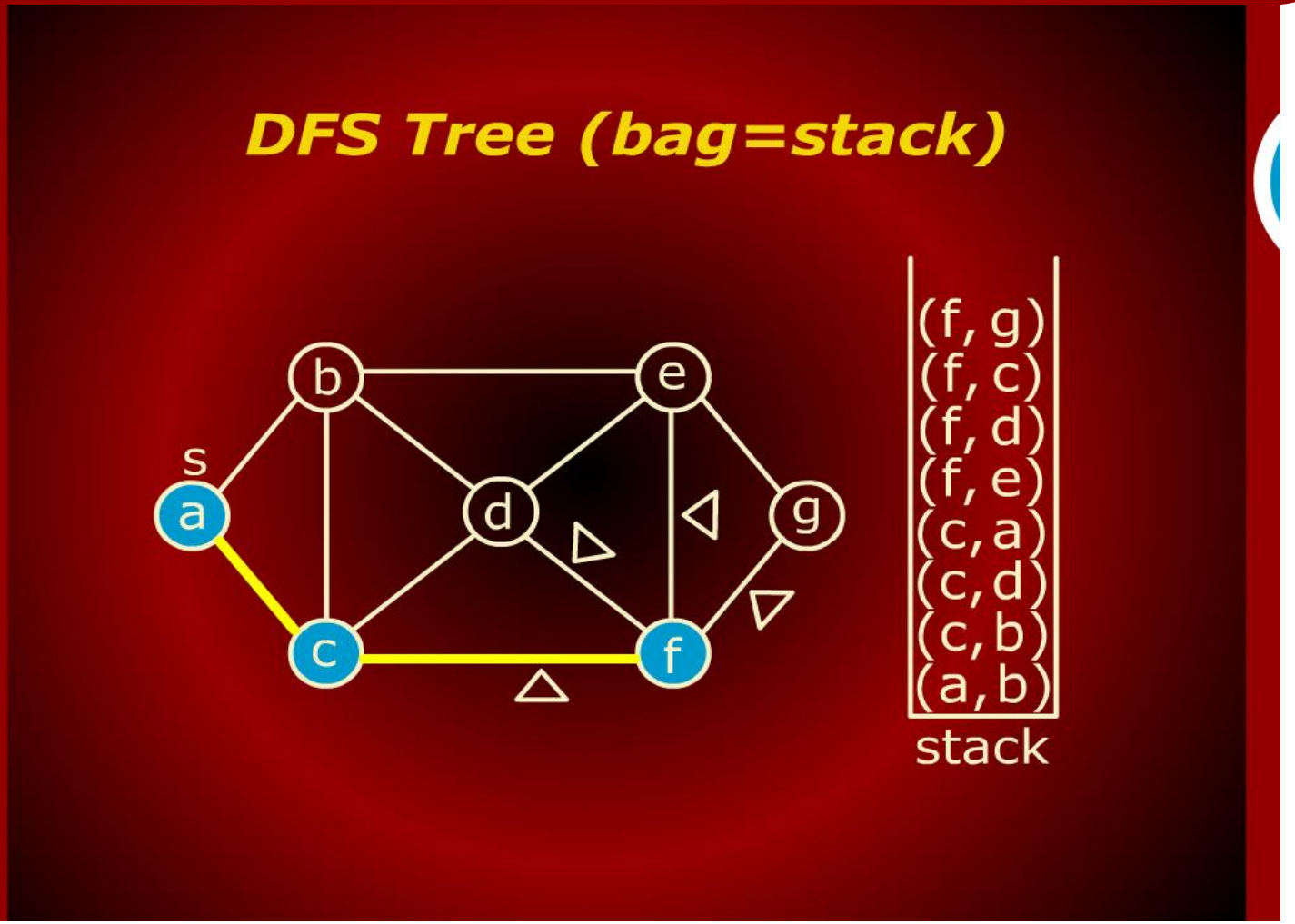
DFS Tree (bag=stack)



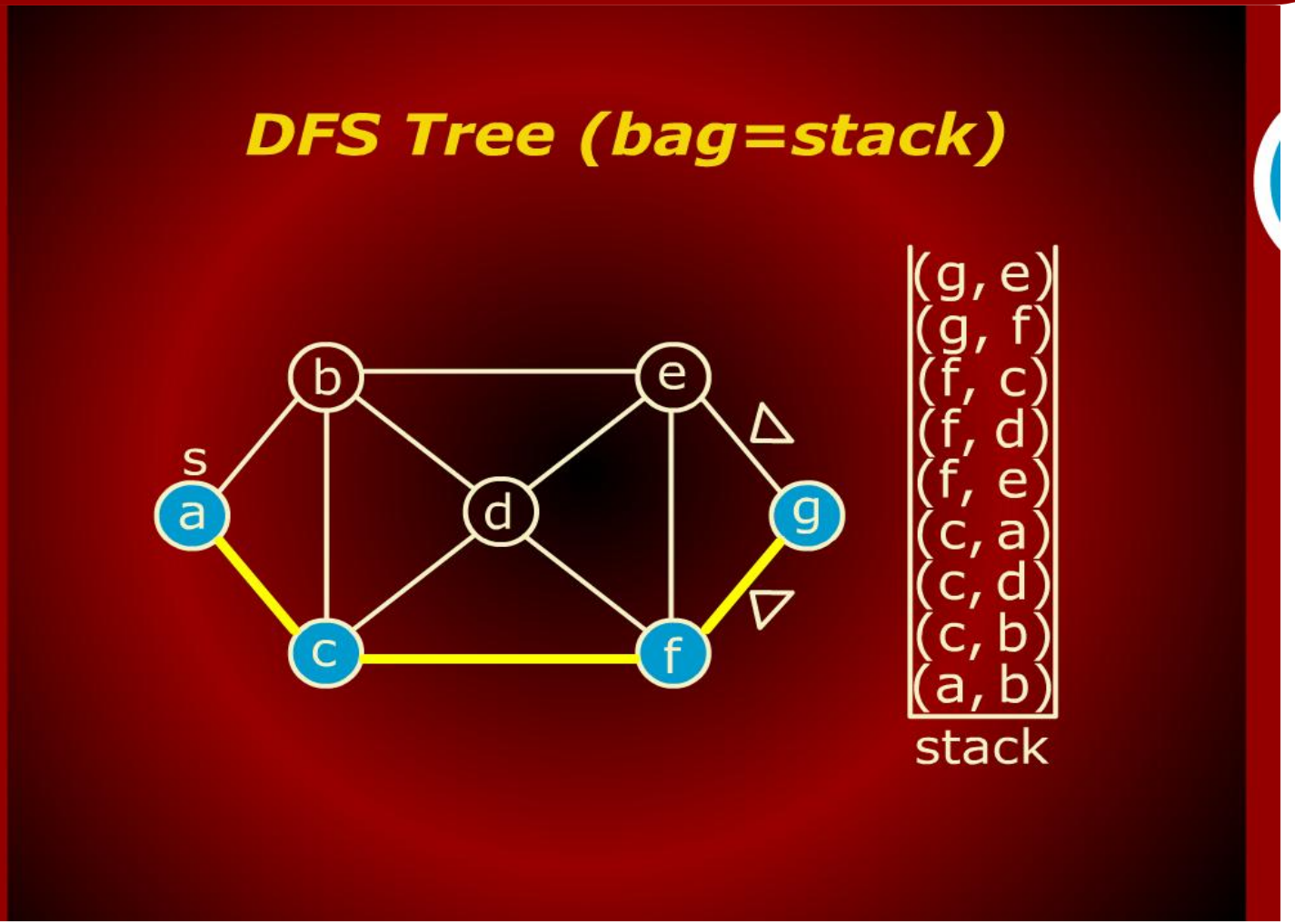
DFS Tree (bag=stack)



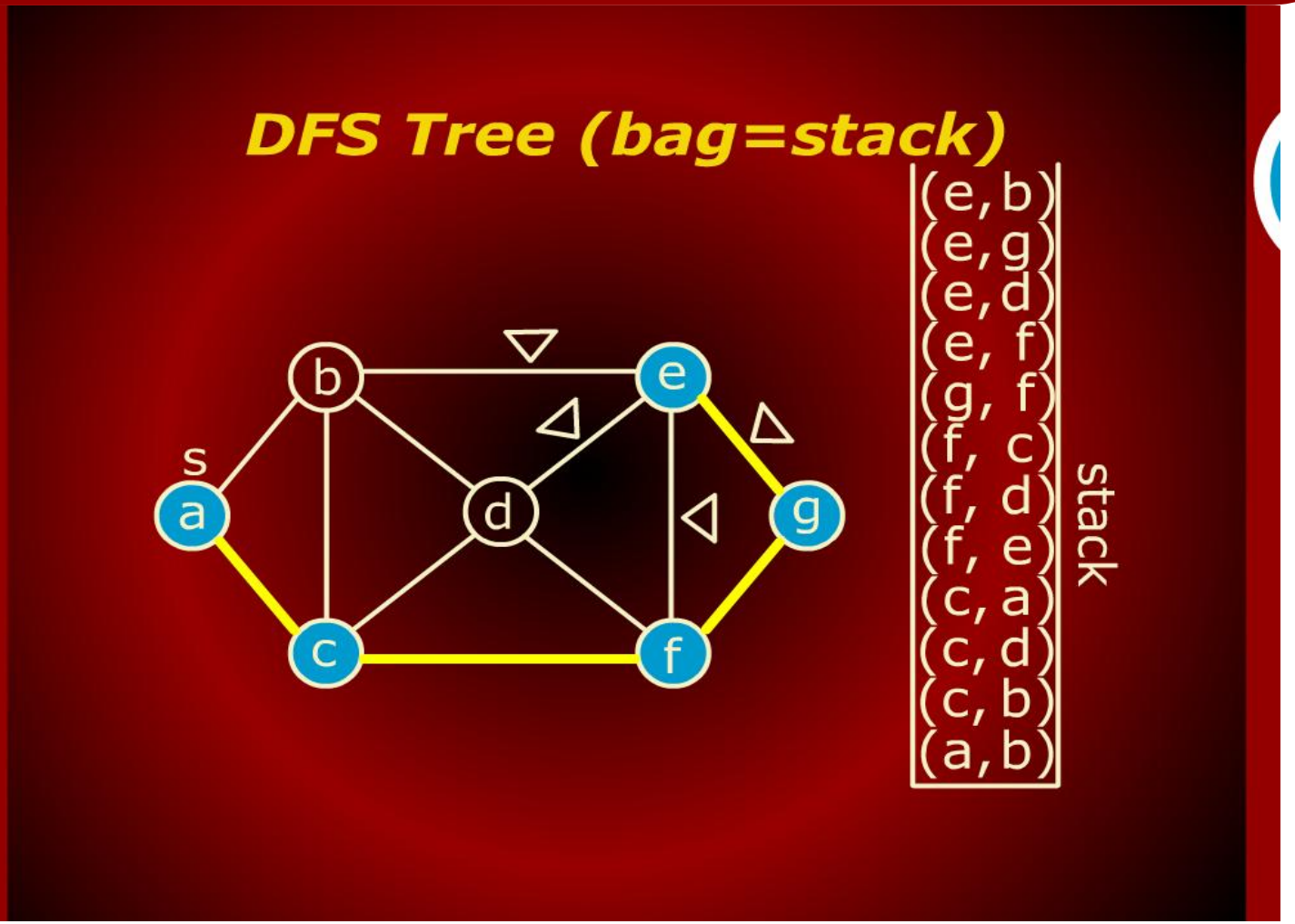
DFS Tree (bag=stack)



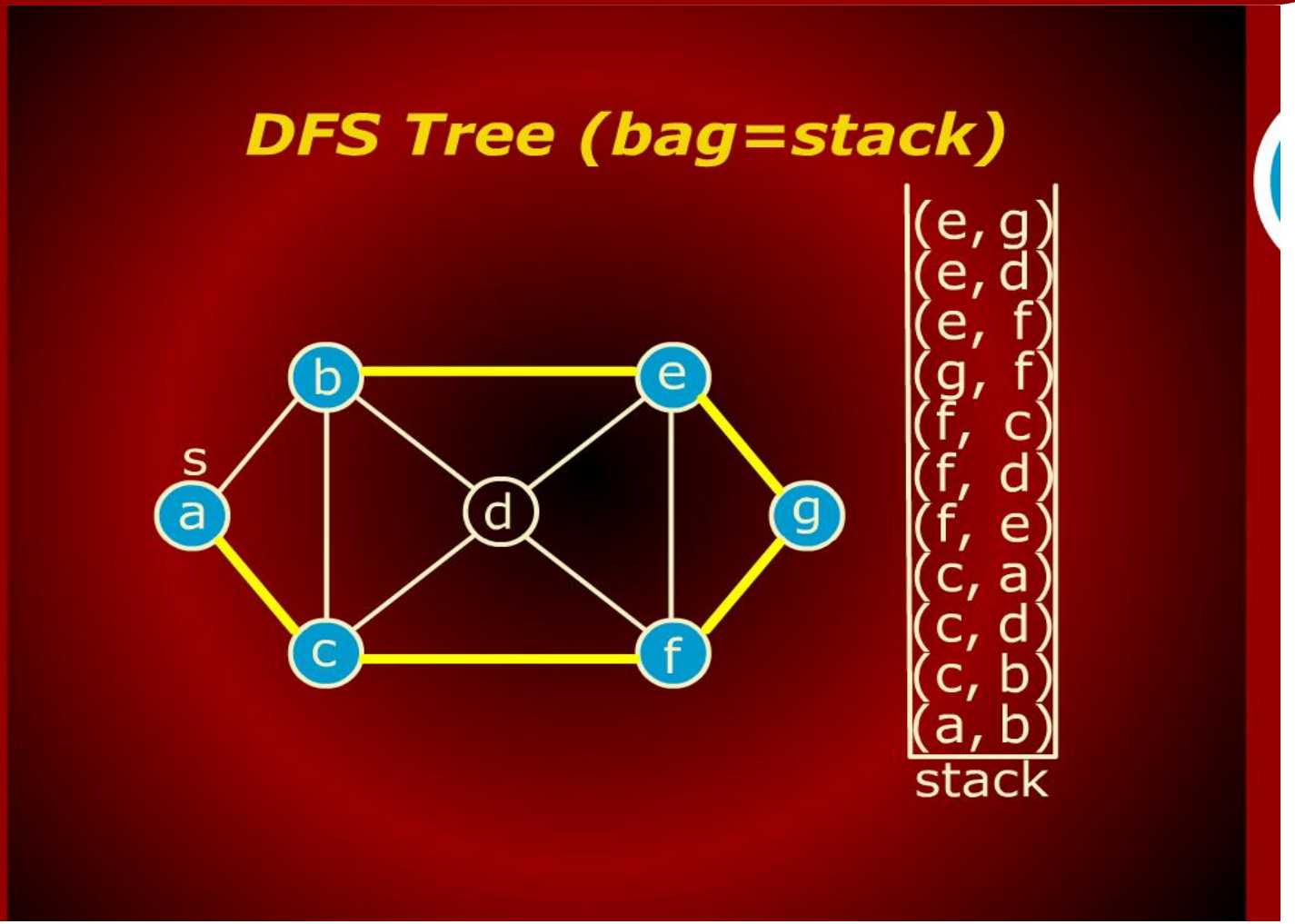
DFS Tree (bag=stack)



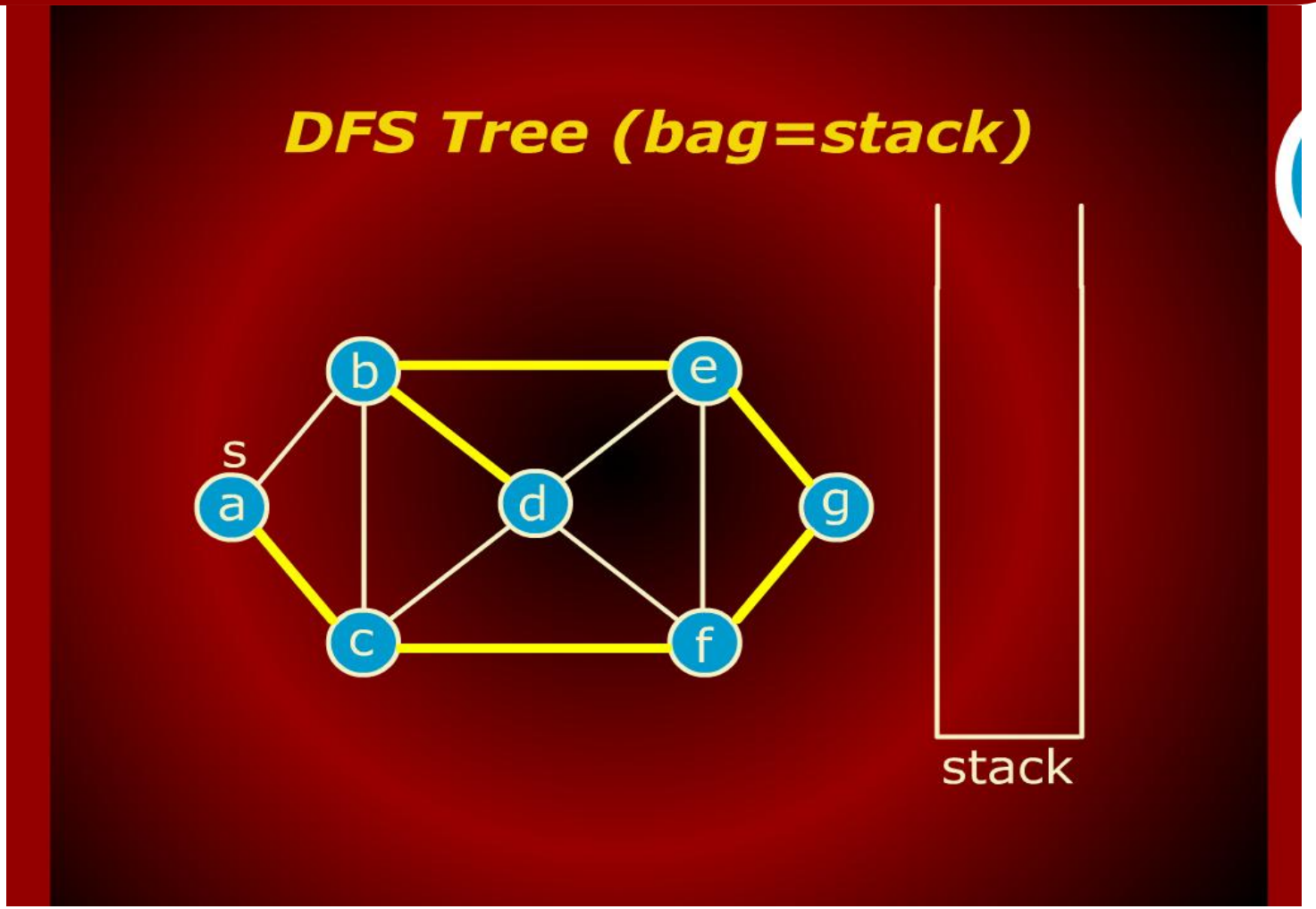
DFS Tree (bag=stack)



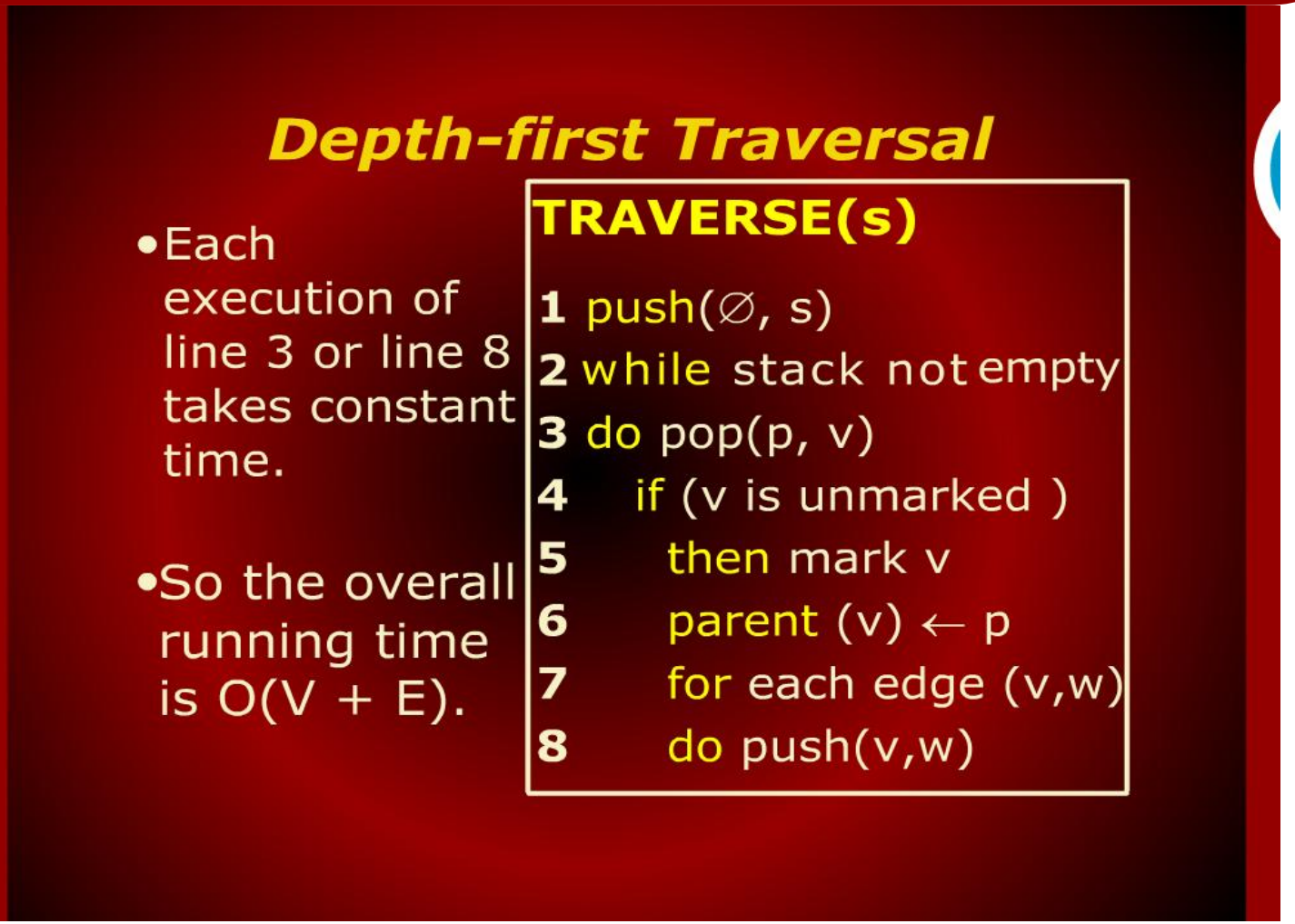
DFS Tree (bag=stack)



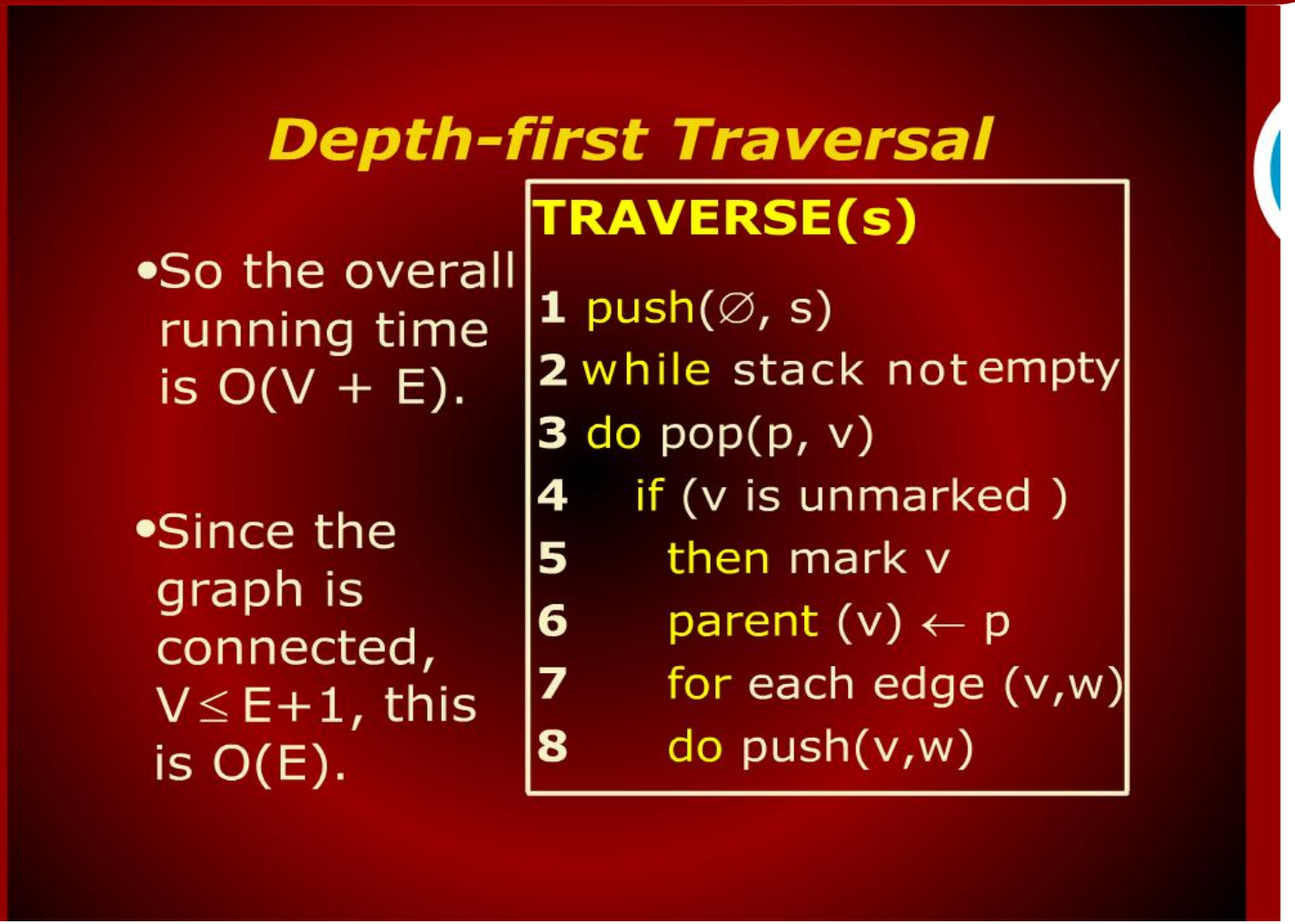
DFS Tree (bag=stack)



Depth-first Traversal



Depth-first Traversal



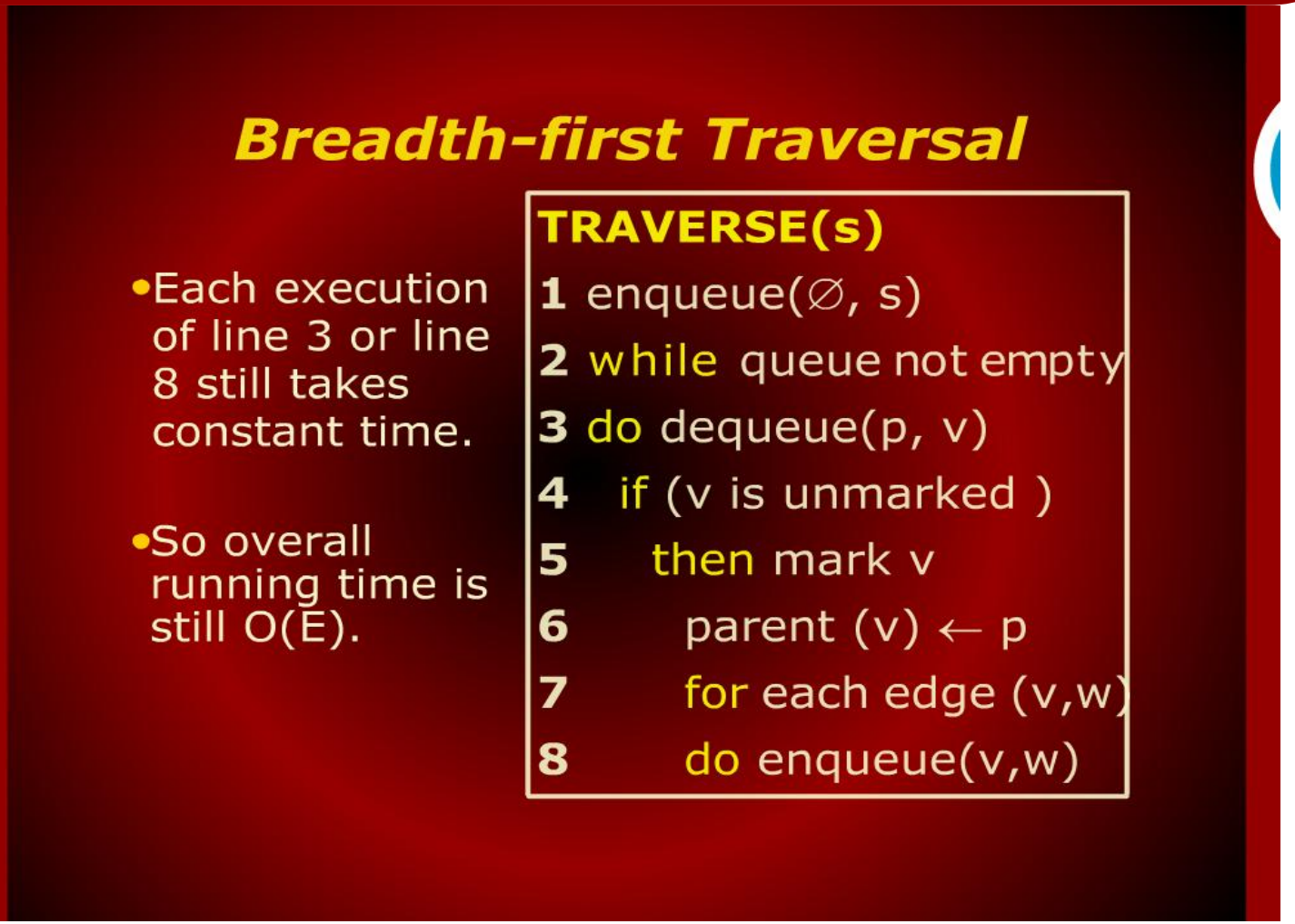
Breadth-first Traversal

Breadth-first Traversal

- If we implement the bag by using a queue,wehave breadth-first search (BFS).
- Each execution of line 3 or line 8 still takes constant time.

```
TRAVERSE(s)
1 enqueue(∅, s)
2 while queue not empty
3 do dequeue(p, v)
4   if (v is unmarked )
5     then mark v
6          parent (v) ← p
7          for each edge (v,w)
8          do enqueue(v,w)
```

Breadth-first Traversal



DFS,BFS with Adjacency Matrix

DFS, BFS With Adjacency Matrix

- If the graph is represented using an adjacency matrix, the finding of all the neighbors of vertex in line 7 takes O(V) time.

```
TRAVERSE(s)
1 put (∅, s) in bag
2 while bag not empty
3 do take(p, v) from bag
4    if (v is unmarked )
5       then mark v
6            parent (v)←p
7            for each edge (v,w)
8            do  put (v,w) in bag
```

DFS,BFS with Adjacency Matrix

DFS, BFS With Adjacency Matrix

- Thus depth-first and breadth-first take O(V2) time overall.

```
TRAVERSE(s)
1 put (∅, s) in bag
2 while bag not empty
3 do take(p, v) from bag
4   if (v is unmarked )
5     then mark v
6          parent (v)←p
7          for each edge (v,w)
8          do  put(v,w) in bag
```

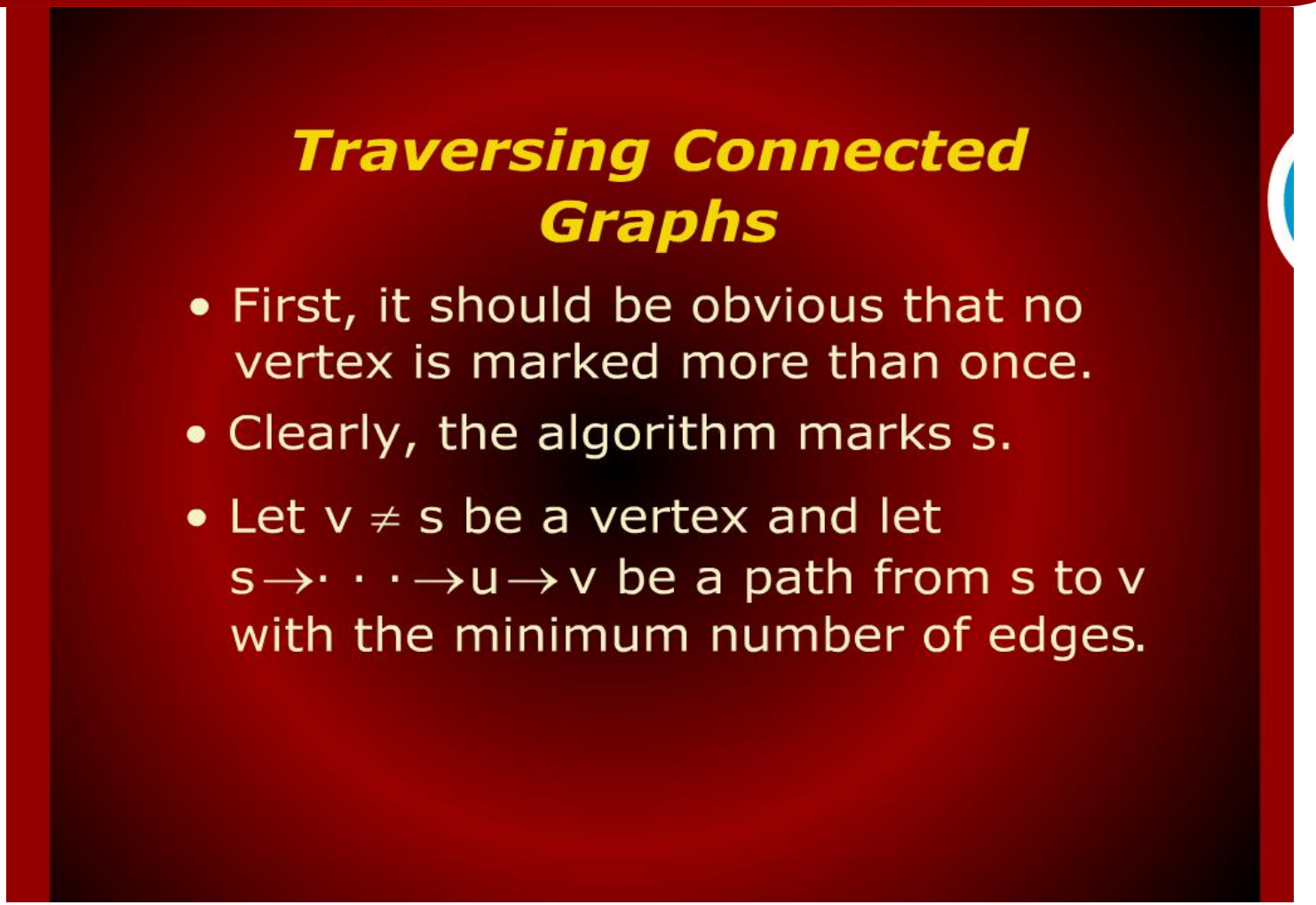
Traversing Connected Graphs

Traversing Connected Graphs

Why does DFS or BFS yield a tree?

- Traverse(s) marks every vertex in any connected graph exactly once and the set of edges (v, parent(v)) with parent(v) $\neq \varnothing$ form a spanning tree of the graph.

Traversing Connected Graphs



Traversing Connected Graphs

Traversing Connected Graphs

- Since the graph is connected, such a path always exists.
- If the algorithm marks u, then it must put (u, v) into the bag, so it must take (u, v) out of the bag at which point v must be marked.
- Thus, by induction on the shortest-path distance from s, the algorithm marks every vertex in the graph.

Traversing Connected Graphs

Traversing Connected Graphs

- Call an edge (v, parent(v)) with parent(v) $\neq \varnothing$, a parent edge.
- For any node v, the path of parent edges v $\rightarrow$ parent(v) $\rightarrow$ parent (parent(v)) $\rightarrow$. . . eventually leads back to s.
- So the set of parent edges form a connected graph.

Traversing Connected Graphs

Traversing Connected Graphs

- Clearly, both end points of every parent edge are marked, and the number of edges is exactly one less than the number of vertices.
- Thus, the parent edges form a spanning tree.

DFS – Timestamp Structure

DFS - Timestamp Structure

- As we traverse the graph in DFS order, we will associate two numbers with each vertex.
- When we first discover a vertex u, store a counter in d[u].
- When we are finished processing a vertex, we store a counter in f[u].
- These two numbers are time stamps.

DFS – Timestamp Structure

DFS - Timestamp Structure

Consider the recursive version of depth-first traversal

```
DFS(G)
1 for (each u ∈ V)
2 do color[u] ← white
3      pred[u] ← nil
4 time ← 0
5 for each u ∈ V
6 do if (color[u] = white)
7         then DFSVISIT(u)
```

DFS – Timestamp Structure

Model of Computation

Depth-first Search